

Ghost Social — Architecture Technique

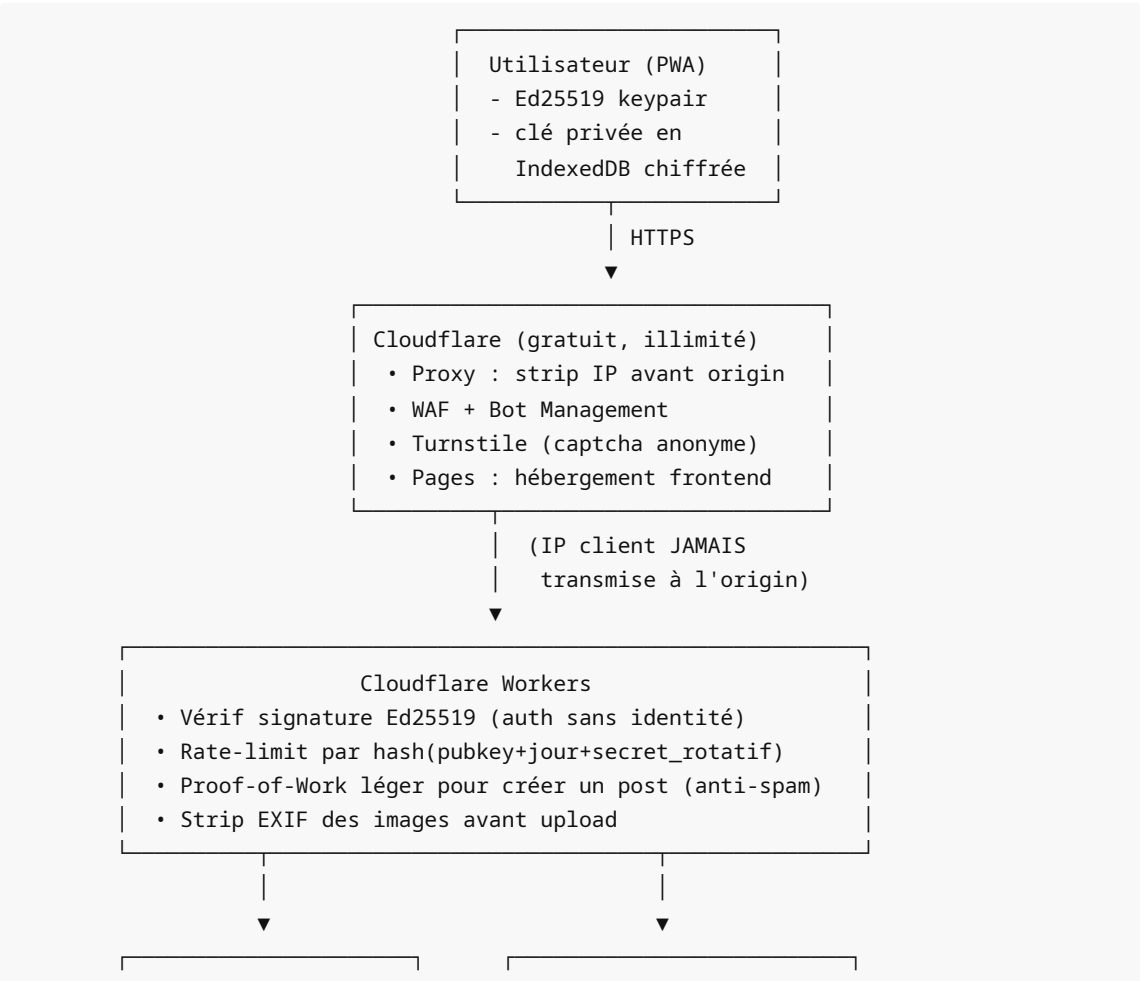
Réseau social fantôme. Anonymat total, budget ~0€, trajectoire de scale claire.

1. Le problème, tel qu'on l'a formulé

Contrainte	Implication technique
Anonymat total	Pas d'email, pas de téléphone, pas d'IP loggée, pas de fingerprint. Le serveur ne doit jamais pouvoir ré-identifier un utilisateur — même sous contrainte légale.
Budget quasi nul	0€/mois jusqu'aux premiers 10k utilisateurs actifs. Pas de CB requise pour les services critiques.
Réseau social	Écritures fréquentes, feed temps réel, médias, modération, scale potentiellement exponentiel.

Ces deux contraintes (anonymat + coût) sont **contradictoires** avec les réflexes habituels : pas de Firebase Auth (email requis), pas d'S3 (CB + egress payant), pas de Sentry (il logge les IPs).

2. Schéma d'architecture



Supabase (free)	Cloudflare R2 (free)
• Postgres 500 MB	• 10 GB, egress=0€
• Row-Level Sec.	• Médias post-stripping
• Realtime (WS)	
• pgBouncer inclus	

3. Choix techniques — le « Pourquoi »

3.1 Authentification : keypair Ed25519 côté client

Décision. L'utilisateur génère une paire de clés Ed25519 dans son navigateur. Le serveur ne stocke que le `hash(pubkey)` . Chaque requête est signée.

Pourquoi plutôt qu'une alternative ?

Option	Verdict	Raison du rejet
Email + mot de passe	✗	Email = identifiant. Rompt l'anonymat.
OAuth (Google/Apple)	✗	Transfère l'identité au fournisseur. Pire.
Téléphone + OTP	✗	Coût SMS + numéro = identité réelle.
UUID anonyme stocké serveur	✗	Le serveur « connaît » l'utilisateur. Sous subpoena, on balance.
Ed25519 client-side	✓	Le serveur ne peut techniquement pas ré-identifier. Inspiré de Nostr, protocole éprouvé en prod (~993k profils utilisateurs avec contact list, 33M de pubkeyes au total en août 2024) [10].

Preuve d'anonymat. Le serveur ne voit jamais de donnée corrélable à une personne physique. Même un dump complet de la BDD + des logs Cloudflare ne permet pas de remonter à un individu (Cloudflare ne nous transmet pas l'IP, et on ne demande jamais de PII).

Trade-off assumé. Perte de clé = perte de compte. Pas de « mot de passe oublié ». C'est **volontaire** : toute procédure de récupération ouvrirait une faille d'identification. On guide l'utilisateur vers une sauvegarde de phrase mnémonique (BIP-39, 12 mots = 128 bits d'entropie, standard utilisé par toute l'industrie crypto wallet) [11].

3.2 Edge : Cloudflare (Proxy + Workers + Pages + R2 + Turnstile)

Décision. Tout le edge sur Cloudflare.

Pourquoi ?

- **Anonymat** : Cloudflare permet de retirer l'IP avant transmission à l'origin via le **Managed Transform « Remove visitor IP headers »** — le header `CF-Connecting-IP` (et ses équivalents `True-Client-IP` , `X-Forwarded-For`) est strippé côté edge [6]. Notre backend ne voit **jamais** d'IP client.
- **Coût** : plan gratuit couvre Pages (bandwidth illimité, requêtes static illimitées) [2], Workers (100 000 req/jour, 10 ms CPU/invoction) [1], R2 (10 GB de stockage + **egress = 0 \$ toujours**) [3], Turnstile (vérifications illimitées, sans cookies, conforme RGPD) [4].

- **DDoS** : inclus gratuitement. Critique pour un service anonyme, cible privilégiée.

Comparaison honnête.

Alternative	Problème
Vercel + S3	S3 egress payant dès le jour 1. Vercel bandwidth 100 GB puis payant.
Netlify + Firebase	Firebase = Google, exige CB, logue tout.
VPS Hetzner CX23 (3,49 €/mois, 20 TB de transfert inclus en EU) [13]	Excellent rapport qualité/prix, mais pas 0€ et on gère soi-même DDoS, TLS, scaling. On le garde en plan B (voir §5).

Trade-off. Dépendance à un acteur unique (Cloudflare). Mitigation : stack portable (Postgres standard, R2 est S3-compatible, Workers exportables vers Node). Un vendor-lockout coûterait ~1 semaine de migration, pas une ré-architecture.

3.3 Base de données : Supabase (Postgres managé)

Pourquoi Postgres et pas NoSQL ? Un feed social avec follows, threads, reactions est un graphe relationnel. Les JOIN et index composites sur Postgres battent toute solution document pour ce pattern.

Pourquoi Supabase plutôt que...

Option	Rejet
MongoDB Atlas free	512 MB, pas de realtime inclus, pas de RLS.
Firebase	Déjà rejeté (identité Google).
Neon / PlanetScale free	Bon Postgres mais sans realtime ni storage intégré → il faut recâbler.
Supabase free	500 MB Postgres + 50 000 MAU auth (qu'on n'utilise PAS) + Realtime + 1 GB file storage + 500k invocations Edge Functions [8]. Pooler Supavisor inclus pour tous les projets, y compris free [9]. Tout-en-un.

Row-Level Security. Chaque row a un `author_pubkey_hash`. Les politiques RLS garantissent qu'une requête ne peut lire/écrire que ce que la signature autorise. Même une injection SQL est contenue par RLS.

Trade-off. 500 MB se remplit. Estimation : ~1 KB par post → 500k posts. On active la compression TOAST et on archive les posts > 6 mois dans R2 (format Parquet, interrogeable via DuckDB si besoin).

3.4 Modération sans identité (le problème dur)

Le dilemme. Sans identifiants, comment bannir un troll qui crée 1000 comptes ?

Stratégie en couches :

1. **Proof-of-Work client** pour créer un compte et pour poster (hashcash-style, ~200 ms CPU). Invisible pour un humain, coûteux pour un bot qui en crée 10k.
2. **Turnstile** (Cloudflare, anonyme, **ne lit ni n'écrit aucun cookie**, conforme ePrivacy/RGPD/CCPA) [4] sur les actions sensibles.
3. **Rate-limit par `hash(pubkey + jour + secret_serveur_rotatif)`** : on compte sans savoir qui.

4. **Modération communautaire** : seuil de signalements → masquage automatique + review par modérateurs bénévoles qui ne voient que le contenu, pas l'auteur.
5. **Shadowban réversible** : on marque le hash de pubkey, pas la personne.

Honnêteté intellectuelle. Ce système **ne rend pas le spam impossible**, il le rend **économiquement non viable**. On accepte qu'un attaquant déterminé avec des ressources puisse nuire ; on vise à éliminer 99 % du bruit.

3.5 Observabilité sans logs personnels

- **Métriques** : Grafana Cloud free (10 000 séries de métriques actives, 50 GB de logs, 14 jours de rétention) [15]. Uniquement des compteurs agrégés, jamais de payload.
- **Pas de Sentry** : il logue IPs et stack traces avec données utilisateur. On utilise **GlitchTip** : open-source, **API-compatible avec les SDK Sentry** (donc drop-in), tient sur 2 vCPU / 2 GB RAM (vs 32 GB pour Sentry self-hosted) [14]. Hébergé sur le VPS de plan B avec scrubbing agressif des PII.
- **Analytics** : Cloudflare Web Analytics — **sans cookies, sans localStorage, sans fingerprinting** (ne lit pas l'IP ni le User-Agent à des fins analytiques), gratuit [5].
- **Stripping EXIF côté serveur (Worker)** avant upload R2 : les EXIF contiennent GPS + modèle d'appareil, considérés comme données personnelles sous RGPD [16].

4. Vérification contre les contraintes

Contrainte	Comment elle est tenue
Anonymat total	(a) Pas de PII collectée. (b) IP stoppée à Cloudflare. (c) Auth cryptographique sans identifiant. (d) Serveur incapable techniquement de ré-identifier.
Budget ~0€	Toute la stack tient dans les plans gratuits jusqu'à ~10k MAU. Aucun service sur CB obligatoire avant seuil critique.

Coût mensuel réel au démarrage : 0,00 €. Nom de domaine optionnel (~12€/an sur Porkbun) — sinon `.pages.dev` gratuit.

5. Scalabilité — « Si le trafic double demain ? »

Palier	Goulet d'étranglement	Solution	Coût
0 → 10k MAU	Aucun	Rien à faire	0 €
10k → 50k MAU	Postgres 500 MB	Supabase Pro (8 GB, pgBouncer, backups)	25 \$/mois
50k → 500k MAU	Workers 100k req/jour	Workers Paid	5 \$/mois + 0,30 \$/M req
500k → 5M MAU	Postgres writes	Read-replicas + partitionnement par <code>author_pubkey_hash</code>	~200 \$/mois
5M+ MAU	Monolithe Postgres	Sharding applicatif + Citus ou migration vers CockroachDB	Discuter avec un investisseur 😊

Le point clé. Chaque palier est un **changement de plan**, pas une ré-architecture. La stack supporte nativement la croissance.

Plan B de sortie de Cloudflare. Si Cloudflare devient hostile (pricing, policy) : Workers → Node/Bun sur Hetzner CX23 (3,49 €/mois, 20 TB de transfert inclus en EU) [13], R2 → Backblaze B2 (S3-compatible, egress B2 → Cloudflare gratuit via Bandwidth Alliance, partenariat formalisé) [12], Pages → Caddy sur le même VPS. Migration estimée : 5-7 jours.

6. Compromis assumés (l'honnêteté)

1. **Pas de récupération de compte.** Assumé — c'est le prix de l'anonymat réel.
 2. **Modération imparfaite.** On dissuade, on ne bloque pas absolument.
 3. **Dépendance Cloudflare forte** au démarrage. Mitigée par la portabilité de la stack.
 4. **Pas de features « social graph personnalisé »** lourdes (recommandations ML) tant qu'on est gratuit — coûteux en compute. On livre d'abord un feed chronologique + tags.
 5. **Latence Workers en edge ≠ latence base centralisée.** Supabase a des régions ; on choisit eu-west-1 si public principal en Europe. Trade-off latence/coût assumé (multi-région = payant).
-

7. Questions agressives anticipées — réponses prêtes

« **Pourquoi pas juste Firebase, c'est gratuit et plus simple ?** » Firebase exige un compte Google côté utilisateur ou côté dev. Le TOS l'autorise à utiliser les métadonnées. L'anonymat est incompatible avec Firebase Auth. Sur une contrainte « anonymat total », Firebase est disqualifié, point.

« **Ed25519 côté client, c'est mignon, mais 99 % des users vont perdre leur clé.** » Vrai risque. Mitigation : (1) mnémorique BIP-39 à la création, (2) export/import dans les settings, (3) sync optionnel entre devices via QR code. Mais on **refuse** toute récupération serveur — ce serait un backdoor d'identification.

« **Cloudflare Workers a un cold start, Supabase free s'endort. Vous servez qui à 3h du matin ?** » Workers : startup <1 ms en moyenne (plafond 5 ms), grâce aux V8 isolates — c'est ~~100x~~ 100x plus rapide que Lambda** [7]. Honnêteté : **Supabase free se met effectivement en pause après 7 jours d'inactivité** [8] — ce n'est pas un blocage à 3h du matin (un seul utilisateur actif suffit à garder le projet éveillé), mais on pose un cron Workers gratuit qui ping le projet toutes les 24 h pour blinder. Si trafic réel, jamais de pause.

« **Proof-of-work, c'est hostile au batteries des mobiles.** » 200 ms sur un CPU de smartphone 2020 = inaudible en UX. Coût énergétique ridicule vs un scroll TikTok. Alternative si on change d'avis : Turnstile seul, mais moins résistant aux fermes de bots humaines.

« **Votre BDD tient 500k posts. Et après ?** » Palier prévu \$5. Archivage froid vers R2 en Parquet. À 500k posts, on a soit échoué (pas de users), soit réussi (on paye 25 \$/mois sans sueur).

« **Sous réquisition judiciaire, que donnez-vous ?** » Des hashes de pubkeys et du contenu public. Aucune donnée permettant de ré-identifier un individu — parce qu'on n'en a pas. C'est by design, pas une promesse.

8. Ce qu'on livrerait en MVP (2 semaines, 1 dev)

- ☒ Génération keypair + mnémorique à l'onboarding
- ☒ Post texte + image (EXIF stripped)
- ☒ Feed chronologique + par tag
- ☒ Reactions + reply threads
- ☒ Signalement + seuil de masquage

- ☒ PWA installable, offline-first (cache IndexedDB)

Hors scope v1 : DM (nécessite E2EE proprement pensé), vidéo (coût stockage), recommandations ML.

9. Sources (toutes consultées le 2026-04-16)

[1] Cloudflare Workers — Pricing & limits free plan :

<https://developers.cloudflare.com/workers/platform/pricing/> [2] Cloudflare Pages — Limits (bandwidth illimité, requêtes static illimitées) : <https://developers.cloudflare.com/pages/platform/limits/> [3] Cloudflare R2 — Pricing (10 GB free, egress = 0 \$) : <https://developers.cloudflare.com/r2/pricing/> [4] Cloudflare blog — « Turnstile is free for everyone » (illimité, sans cookie, RGPD) : <https://blog.cloudflare.com/turnstile-ga/> [5] Cloudflare Web Analytics — docs (cookieless, sans fingerprinting) : <https://developers.cloudflare.com/web-analytics/about/> [6] Cloudflare HTTP headers — Managed Transform « Remove visitor IP headers » : <https://developers.cloudflare.com/fundamentals/reference/http-headers/> [7] Cloudflare blog — « Eliminating cold starts with Cloudflare Workers » : <https://blog.cloudflare.com/eliminating-cold-starts-with-cloudflare-workers/> [8] Supabase — Pricing officielle (500 MB / 50k MAU / pause après 7 jours) : <https://supabase.com/pricing> [9] Supabase blog — « Supavisor 1.0: a scalable connection pooler » (inclus pour tous les projets) : <https://supabase.com/blog/supavisor-postgres-connection-pooler> [10] Wikipedia — Nostr (protocole Ed25519, ~993k profils en 2024) : <https://en.wikipedia.org/wiki/Nostr> [11] Trezor — « What is BIP39 ? » (mnémonique 12 mots, 128 bits d'entropie) : <https://trezor.io/learn/advanced/standards-proposals/what-is-bip39> [12] Backblaze blog — « B2/Cloudflare Partnership Offers Free Data Transfer » (Bandwidth Alliance) : <https://www.backblaze.com/blog/backblaze-and-cloudflare-partner-to-provide-free-data-transfer/> [13] Hetzner Cloud — Pricing officiel + Price Adjustment 2026-04-01 : <https://www.hetzner.com/cloud/pricing/> et <https://docs.hetzner.com/general/infrastructure-and-availability/price-adjustment/> [14] GlitchTip — site officiel + comparaison ressources vs Sentry self-hosted : <https://glitchtip.com/> et <https://dev.to/selfhostingsh/glitchtip-vs-sentry-206o> [15] Grafana Cloud Free Tier — 10k metrics + 50 GB logs + 14 j rétention : <https://grafana.com/products/cloud/free-tier/> [16] EDUCAUSE Review — « Privacy Implications of EXIF Data » (GPS, identification, RGPD) : <https://er.educause.edu/articles/2021/6/privacy-implications-of-exif-data>